

Car Detection Based Algorithm For Automatic Parking Space Detection

Raj Patel

Computer Engineering
California State University
Sacramento, USA
rajkirtibhaipatel@csus.edu

Praveen Meduri

Electrical and Electronic Engineering
California State University
Sacramento, USA
praveen.meduri@csus.edu

Abstract—In this paper, we present an object detection based algorithm to automatically map the parking spaces in a parking lot, instead of manually mapping them. The work addresses an important gap in the recent computer vision based artificial intelligence techniques to build smart parking systems. We test our approach with two of the most popular object detectors, Faster R-CNN and YOLOv4. Our results show that our approach decreases the human effort needed by up to a compelling 90%. We show that the percentage of the available parking spots that are automatically detected through our approach accumulates over time and, in theory, can approach a 100%, on a day when all the parking spots are fully occupied. In other words, the approach is designed to have its highest performance over a busy parking lot during the busiest time.

Keywords—CNN, Smart Parking Systems, Artificial Intelligence, Computer Vision, Machine Learning

I. INTRODUCTION

The task of finding a parking spot in a busy urban area is time-consuming. The process of blindly cruising around for a suitable parking spot entails significant human effort and leaves the driver with a very unpleasant and stressful experience. The financial loss incurred in this inefficient process can be staggering [1]. This problem is only to be expected to compound as our world becomes more and more urban [2].

Additionally, with increasing acceptance of autonomous vehicles [3], extracting reliable information regarding the layout of a parking lot and the occupancy of each parking space, by means of a scalable and economical smart technology, becomes vital. This information can guide the vehicles to the exact location of the nearest available parking space, automatically.

Efforts to address this problem have proposed Artificial Intelligence and Computer Vision based solutions towards developing Smart Parking Systems [4]. An effective smart parking solution should have real-time information about the occupancy status of all the parking spots. This parking information should be compiled into a central repository for processing. Relevant information about available spots should be gleaned and distributed to users in a “real-time” and user friendly manner. All of these tasks are organized logically into 5 modules of the SParkSys framework [5].

Computer Vision based smart parking solutions have gained much attention recently. [4] shows that camera sensors

are relatively cost effective, provide multiple detection using a single sensor and are non-intrusive to install. Advances in deep learning and convolutional neural network (CNN) have made it feasible to extract high level features from camera images. They provide excellent accuracy at performing human-level, complex cognitive tasks like object detection, scene interpretation and instance segmentation. Recent works [1], [6], [7] used CNN to accurately predict the occupancy status of parking spots in a parking facility. However most of the works published so far, including the afore mentioned, use a manual process to map each individual parking spot from the image of a parking lot. The parking spots can then be classified as occupied or empty.

The process of mapping all parking spots in a parking facility requires a human to painstakingly label boxes around each parking spot in the image. Each image can contain up to 150-200 parking spaces and thus adds a significant overhead that limits the scalability of a smart parking solutions for large parking facilities.

In this paper, we present a technique to automate the process of mapping a parking lot into its constituent parking spots. Specifically, we will compare a Faster R-CNN [8] and YOLOv4 [9] based object detection algorithm to identify parked cars in a parking lot and then infer parking spots from the location of the parked cars. Our results show that this approach can bring about a compelling 90% reduction in the human effort involved in mapping parking lots. In addition to inheriting the merits of recent object detection schemes, an innate advantage of our approach is that the overall recall of the parking space detection scheme increases with increasing number of the parked cars. We show that the percentage of the available parking spots that are automatically detected through our approach accumulates over time and, in theory, can approach a 100%, on a day when all the parking spots are fully occupied. In other words, the approach is designed to have its highest performance over a busy parking lot during the busiest time.

The rest of the paper is organized in the following manner. Section II presents a general overview of our approach. In section III, we evaluate the performance of the proposed approach. In section IV, discussions and conclusions are presented.

II. METHODOLOGY

In this section, an overview of our approach is provided. The intuition behind this approach is that parked cars remain stationary for a longer period of time than a car which is not parked. If we can track the movement of cars in the image, then we can identify parked cars which will provide the location of parking spots in the image. The task of locating a parking space in the image is called parking space detection.

To achieve parking space detection, an object detection model is used to identify and locate cars in the image, and a custom algorithm is used to track these locations over time to identify parked cars.

A. Car Detection

The input to an object detection is an image. For each object in the image, it outputs a category label which classifies the object and a bounding box which localized the object in the image. A bounding box is a set of 4 co-ordinates (x1, y1, x2, y2) where (x1, y1) represent the top-left point of the box and (x2, y2) represent the bottom-right point.

Faster R-CNN and YOLOv4 models will be trained for car detection. CARPK dataset [10] will be used for training and validation. Early stopping is used to decide the number of training iteration.

Faster R-CNN implementation from Detectron 2 [11] is used, specifically “faster_rcnn_R_50_FPN_3x” from the model zoo. It has a ResNet-50 Feature Pyramid Network [12] as a backbone.

YOLOv4 implementation based on Darknet [13] Framework was used. YOLOv4 is a single stage detector which provides fast and accurate detection [9].

B. Car Tracking

The aim of the car tracking algorithm is to distinguish between stationary cars and moving cars. Each frame of the parking lot is processed sequentially as shown in Fig. 4.

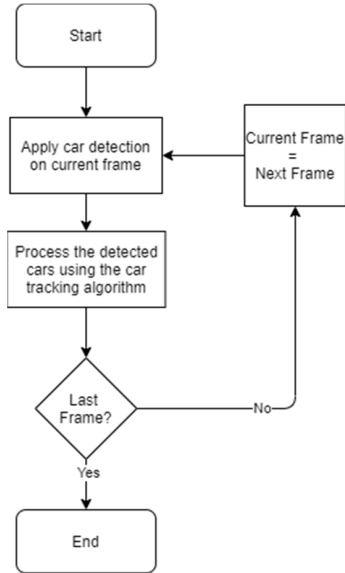


Fig. 1. Flowchart of parking space detection.

$$IoU(Box1, Box2) = \frac{Intersection_Area(Box1, Box2)}{Union_Area(Box1, Box2)} \quad (1)$$

The car tracking algorithm takes as inputs, the bounding boxes for the cars detected by the model in the current frame and compares them with the list of cars currently being tracked, this list will be called potential parking spots. Intersection over Union (IOU) Ratio as shown in (1), is used to compare two bounding boxes. Two distinct boxes with no overlap would have an IoU of 0. Their IoU increases as the amount of their overlap increases, if the intersection is higher than a threshold (IoU Threshold) then the two bounding boxes are considered a match. Three cases are considered by the algorithm.

First case, a new car enters the image as shown by Fig. 1. In this case, the bounding box for the car will not match any of the potential parking spots. Such bounding boxes are added to the list of potential parking spaces.

Second case is of a stationary car (Fig. 2), such car will match its own bounding box from the list of potential parking spot, because the IoU will be high for such cars. In this case, a counter is incremented for this parking spot. If this counter reaches a pre-defined threshold (Frame Threshold), it means that the car has been stationary in that spot for a long enough time and that part of the image is marked as a parking spot.

Third case is of a moving car shown in Fig. 3. The bounding box for the car will be in the list of potential parking spots but it will fail to match any input car from the current frame before it reaches the frame threshold. When this case occurs, the bounding box for that car is removed from the list of potential parking spots. A generic implementation of this algorithm is shown in Fig. 7.

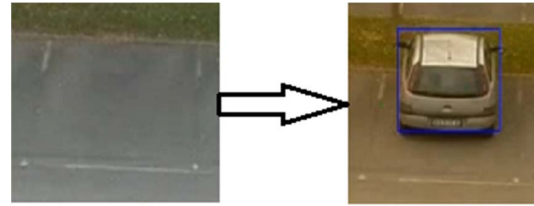


Fig. 2. Case 1: New car

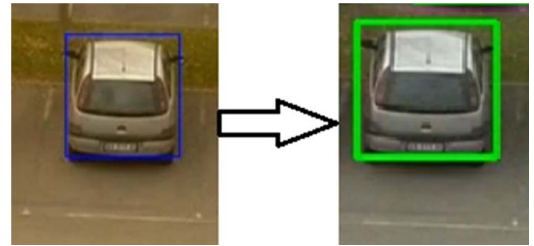


Fig. 3. Case 2: Stationary car



Fig. 4. Case 3: Moving Car

```

for each car in cars:
    match = False
    for each spot in parking_spots, until match = True:
        iou_val = IoU(car, spot[box])
        if iou_val >= IoU_Threshold
            match = True
            if spot[is_valid] == False
                spot[count] = spot.count + 1
                spot[touched] = True
                if spot[count] >= frame_threshold
                    spot[is_valid] = True

    if match == False
        spot[box] = car
        spot[count] = 1
        spot[touched] = True
        spot[is_valid] = False
        add spot to parking spots

for each spot in parking_spots:
    if spot[is_valid] == False and spot[touched] == False
        remove spot from parking_spots
    else
        spot[touched] = False

```

Fig. 5. Generic implementation of car tracking algorithm

A carefully chosen combination of IoU threshold and Frame threshold can be used as an effective boundary condition to distinguish between a moving car and a parked car. If a car is moving its, IoU with its previous bounding box will decrease over time and it will not be able to reach the frame threshold.

If the IoU threshold value is high, the algorithm will become more sensitive to small changes in the location of the bounding box. This could result in detection of redundant bounding boxes for the same parking spot. Also, the IoU threshold value should be high enough to ignore the intersection of adjacent parking spots. Empirical observations show that an IoU threshold in the range of 0.35-0.5 gave the best results.

An appropriate value of frame threshold can be selected based on the frame rate and the static time of the car. For example, if the time interval between each frame is 5 minutes and if we want a car to be static for 20 minutes before declaring the location as a parking spot, then the frame threshold value should be 20 minutes divided by 5 minutes, which is 4.

III. RESULTS

A. Car Detection Model

To evaluate the performance of the car detection model, the test split provided in the CARPK dataset was used and Mean Average Precision (mAP) was used to quantify the model performance. YOLOv4 model trained on MS COCO is tested on CARPK as a baseline. The results in the table 1 show that the car detection model was successful in learning the correct weight to identify features of a car and localize it in the image, as evidenced by the Average Precision (AP). Although the YOLOv4 model performs significantly better than the Faster R-CNN model.

TABLE I. CAR DETECTION MODEL PERFORMANCE

Model	Baseline	Faster R-CNN	YOLOv4
mAP @ (0.5 IoU)	18.57	83.17	97.69

B. Car Tracking

CNRPark+EXT [14] dataset was used to evaluate the tracking algorithm; it contains 320 labelled parking spaces across 9 cameras. 3 busy days were selected from the dataset for each of the 3 weather conditions. A total of 2880 parking spaces were used to evaluate the algorithm.

Predicated parking spaces are marked true positive in two cases. First, if the IoU between the predicted and ground truth parking space is more than 0.5 (Evaluation IoU). Second, if a high portion of the ground truth box is inside the predicted space, by more than 80%. The choice of this number is based on best empirical performance. This additional condition is included because many ground truth labels in the dataset are

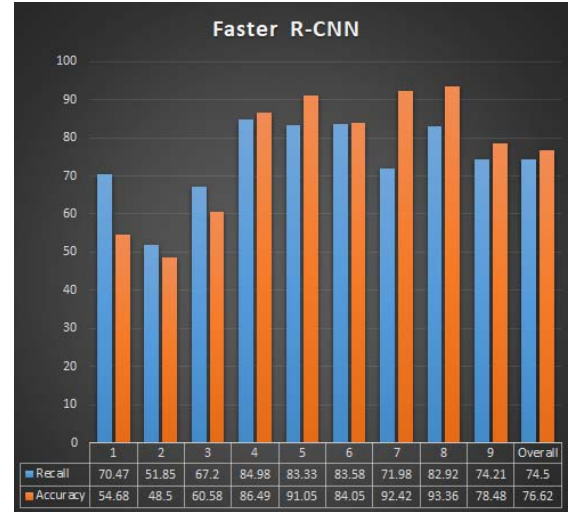


Fig. 6. Parking Space Detection: Faster R-CNN

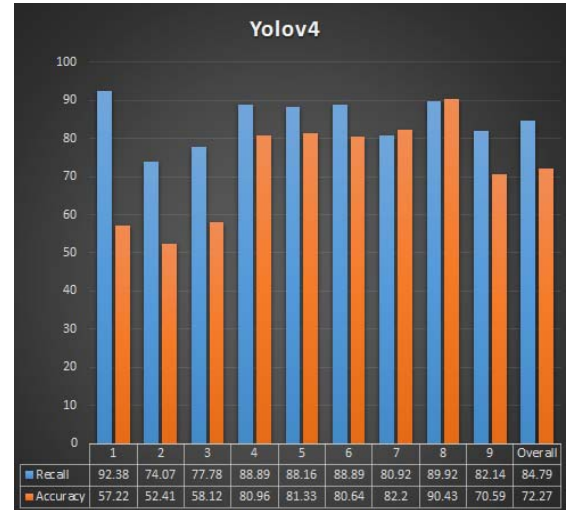


Fig. 7. Parking Space Detection: YOLOv4

much smaller than the actual parking space and IoU comparison is not enough to detect these spaces.

The tracking algorithm was initialized with an IoU threshold of 0.5 and a frame threshold of 3, as the images in the dataset are captured at a 30 minute interval. A car which is parked for 1.5 hour will be marked as parked.

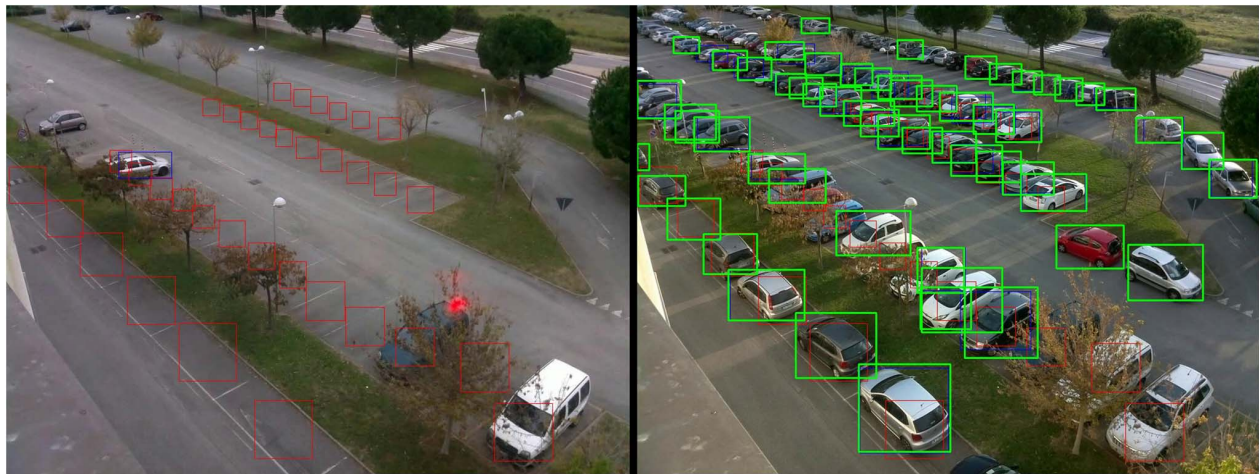


Fig. 8. A comparison of ground truth and predicted boxes. (Red) Ground Truth, (Green) Predicted Parking Space

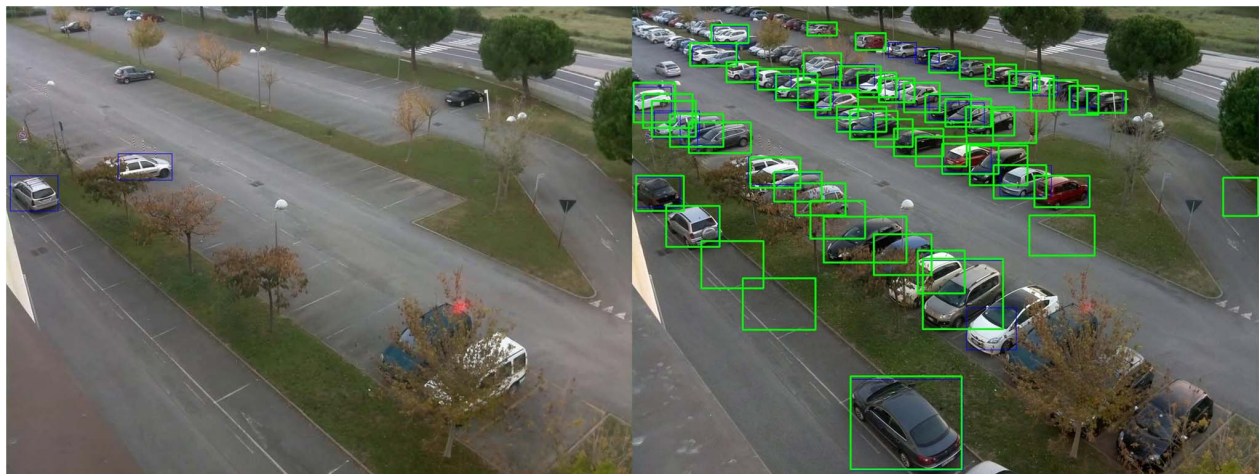


Fig. 9. Camera 1, (Left) before applying the algorithm, (Right) After applying the algorithm. Predicted parking spaces marked by green boxes

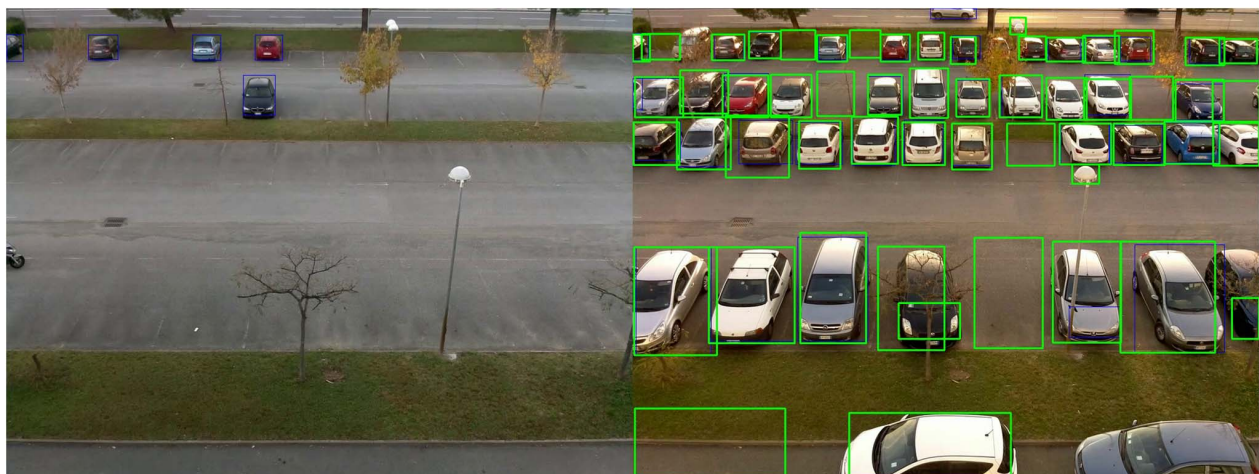


Fig. 10. Camera 8, (Left) before applying the algorithm, (Right) after applying the algorithm. Predicted parking spaces marked by green boxes

Overall recall across 9 cameras is shown in Fig. 5 for the Faster R-CNN model and Fig. 6 for the YOLOv4 model. The YOLOv4 model has a 10% higher recall while having a 4% less precision than the Faster R-CNN model. In terms of computational time, Faster R-CNN finished the test in 591 seconds where as the YOLOv4 model finished it in 251 seconds, 2.4 times faster.

There are two reasons for a parking space to be not detected. Firstly, the object detection model to detect cars in that parking space cannot provide 100% accuracy. Secondly, that parking space is not used by any cars in the images used for prediction.

While the parking space detection scheme inherits the inaccuracies of the underlying car detection scheme; an innate advantage of this approach is that the overall recall of the parking space detection scheme increases with increasing number of the parked cars. In other words, the approach is designed to have its highest performance over a busy parking lot during the busiest time.

The dataset does not label all the parking spaces in the image and leaves some parking spots not labelled properly. But our algorithm was able to correctly identify some of these unlabelled parking spaces as well, as can be seen in Fig. 8. from camera 1, where the red boxes are ground truth and the green boxes are the parking spaces detected by our algorithm. This reduces the apparent precision of our approach, especially across the first 3 cameras, but it actually validates its efficiency because it is a better representation of the ground truth, compared to the data set labels.

Fig. 9 and Fig. 10 above show the images of a parking lot from the CNRPark+EXT dataset, in two different camera viewpoints. On the left are the first frames from the beginning of the day and on the right are the images from the end of the day after processing each frame from that day consecutively. The green boxes show the parking spots identified by the algorithm. Some of the boxes are empty because a parking spot is identified if at some time in the past, a car was parked in that spot. In other words, the recall of the approach accumulates over time and, in theory, can approach a 100%, on a day when all the parking spots are fully occupied.

A proper selection of the threshold values for the IoU and the frame rate determine the accuracy of the car tracking approach. However, these hyper parameters only reduce the accuracy marginally, when they are in a reasonable range. In other words, tuning can be very coarse for the IoU algorithm while still maintaining a reasonable accuracy.

It can be observed from Fig. 5 and Fig. 6 that the performance is consistent across multiple cameras, except for camera 1-3, reasons for which were explained earlier, which alludes to the generalizability of our approach across many different parking lots and camera angles. This phenomenon can be explained based on the observation that our approach depends on the features of a car and not on the features of a parking lot, which means parking spaces can be detected even with faint parking line as can be observe in many test images

IV. DISCUSSION AND CONCLUSIONS

The methods presented in the paper successfully identified a significant number of parking spaces. This approach does not completely automate the process of parking space detection as a human might have to filter and add to the detected parking spaces, but it does aid the process of parking space labelling scientifically and would reduce the time required to label thousands of parking spaces. Our results show that there can be up to a compelling 86% reduction in the human effort.

An accurate car detection model is very important for good performance of the overall algorithm. Improved car detection models can increase the recall of this method. We believe that accuracy of this approach can be improved by a higher frame rate, which would allow for higher frame threshold, but our dataset did not allow to test this scenario. Also, the robustness of the car tracking algorithm can be improved by adding a method to distinguish between each unique car. This can help in the scenario where different cars are at the same position in consecutive frames.

An interesting line of further research to improve the number of detected parking spaces would be to combine our approach with well-established low-level thresholding techniques. For example, it is conceivable that a hybrid approach that attempts to detect parking spaces by using a combination of (i) our car detection approach and also (ii) a Hough transform based parking space detection scheme from the underlying demarcations on the ground could improve the overall performance. However, further work needs to be done to validate this hypothesis.

In conclusion, this paper contributes a method for automatic parking space detection to aid the process of parking space labelling. This approach has its highest performance over a busy parking lot, particularly during a busy time. Our results show that there can be up to a compelling 90% reduction in the human effort in mapping the parking spaces in a parking lot. An innate advantage of this approach is that the overall recall of the parking space detection scheme increases with increasing number of the parked cars. In other words, the approach is designed to have its highest performance over a busy parking lot during the busiest time.

REFERENCES

- [1] G. Bill Yang Cai, Ricardo Alvarez, Michelle Sit, Fábio Duarte, Carlo Ratti, "Deep Learning-Based Video System for Accurate and Real-Time Parking Measurement," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 7693-7701, 2019.
- [2] "World Urbanization Prospects," United Nations Organization, 2014.[Online].Available:<https://www.un.org/en/development/desa/publications/2014-revision-world-urbanizationprospects.html>. [Accessed 07-15-2020].
- [3] National Highway Traffic Safety Administration, "Automated Vehicles for Safety," [Online]. Available: <https://www.nhtsa.gov/technology-innovation/automated-vehicles-safety>. [Accessed 15 07 2020].
- [4] T. Lin, H. Rivano and F. Le Mouél, "A Survey of Smart Parking Solutions," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229-3253, December. 2017.

- [5] T. Eric and M. Praveen, "SParkSys: A Framework for Smart Parking Systems," International Conference on Computational Science and Computational Intelligence, 2017.
- [6] G. Amato, F. Carrara, F. Falchi, C. Gennaro and C. Vairo, "Car parking occupancy detection using smart camera networks and Deep Learning," IEEE Symposium on Computers and Communication, 2016, pp. 1212-1217.
- [7] S. Valipour, M. Siam, E. Stroulia, M. Jagersand, "Parking-stall vacancy indicator system, based on deep convolutional neural networks," in IEEE 3rd World Forum on Internet of Things, 2016.
- [8] S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137-1149, 2017.
- [9] A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," 2020. arXiv:2004.10934.
- [10] H. Meng-Ru, L. Yen-Liang and H. Winston, "Drone-Based Object Counting by Spatially Regularized Regional Proposal Network," IEEE International Conference on Computer Vision (ICCV) 4165-4173, 2017.
- [11] Y. Wu, A. Kirillov, F. Massa, W. Lo and R. Girshick, "Detectron2," 2019. [Online]. Available: <https://github.com/facebookresearch/detectron2>. [Accessed 07-15-2020]
- [12] T. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, "Feature Pyramid Networks for Object Detection," IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 936-944.
- [13] J. Redmon, "Darknet: Open Source Neural Networks in C," [Online]. Available: <http://pjreddie.com/darknet/>. [Accessed 06 10 2020].
- [14] G. Amato, F. Carrara, F. Falchi, C. Gennaro and C. Vairo, "CNRPark+EXT A Dataset for Visual Occupancy Detection of Parking Lots," [Online]. Available: <http://cnrpark.it/>. [Accessed 15 07 2020].